

Siddarth B

CS22B1099_check-1

 Project midsem

Document Details

Submission ID

trn:oid:::3618:138504090

Submission Date

May 11, 2026, 2:02 PM GMT+5:30

Download Date

May 11, 2026, 3:34 PM GMT+5:30

File Name

CS22B1099_check-1.pdf

File Size

620.8 KB

31 Pages

6,179 Words

33,166 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Exclusions

- 3 Excluded Matches

Match Groups

-  **23 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 1%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 23 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 1% Publications
- 2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%
- 2** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%
- 3** Internet
core.ac.uk <1%
- 4** Internet
www.researchgate.net <1%
- 5** Internet
arxiv.org <1%
- 6** Internet
khazna.ku.ac.ae <1%
- 7** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%
- 8** Internet
pdffox.com <1%
- 9** Internet
www.coursehero.com <1%
- 10** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%

11	Internet	docplayer.net	<1%
12	Internet	www.eteks.com	<1%
13	Student papers	Indian Institute of Information Technology, Design and Manufacturing - Kanchee...	<1%
14	Publication	María Camila Guerrero-Giraldo, Miguel Angel Vega-Alonso, Paulo Alonso Gaona-G...	<1%
15	Internet	openaccess.thecvf.com	<1%
16	Student papers	CSU, San Jose State University on 2003-05-13	<1%
17	Student papers	University of Abertay Dundee on 2005-12-01	<1%
18	Publication	K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Co...	<1%

ABSTRACT

This project presents how to re-imagine single-layer perceptron (SLP) learning as a membrane computing problem using the Boolean Numerical P System (BNPS) model. It takes advantage of the parallelism offered by CUDA GPUs — each input training example is assigned its own computational membrane, and all membranes compute simultaneously, so no sequential backpropagation is required. A three-phase CUDA kernel (broadcast, compute, aggregate) was developed to map BNPS directly onto GPU thread blocks, minimising synchronisation overhead.

Experiments were carried out on two datasets, comparing BNPS against standard PyTorch baselines:

1. **Breast Cancer Wisconsin dataset** (569 samples, 30 features): BNPS (100 membranes, 75 steps) achieved **97.37% accuracy** — outperforming the PyTorch SLP (92.98%) and the 4-layer Deep MLP (95.61%) — while training **14.9× to 29× faster** and achieving up to **139.31× GPU speedup** over CPU.
2. **Bitbrains cloud workload dataset** (25 VMs, 94,637 training samples): BNPS at 150 membranes completed training in **104.9ms** — **35.38× faster** than the full-data PyTorch Deep MLP (5→64→32→1, 3710.7ms). At 25 membranes this increases to **139.50× faster** (26.6ms).

Compared to the PyTorch Deep MLP, BNPS used **420× fewer sample-operations** and achieved **13.9× faster inference** (0.1308ms vs. 1.8140ms on 23,671 test samples). These findings confirm that the membrane computing framework offers real advantages in both speed and accuracy over standard GPU-accelerated baselines.

KEYWORDS: Membrane Computing; Numerical P Systems; CUDA; GPU Acceleration; Parallel Learning; Single-Layer Perceptron; Cloud Workload Prediction; Computational Efficiency; Edge Inference.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
LIST OF TABLES	v
LIST OF FIGURES	vi
ABBREVIATIONS	vii
NOTATION	viii
1 INTRODUCTION	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objectives	2
2 LITERATURE REVIEW AND BACKGROUND	3
2.1 Background	3
2.1.1 Artificial Neural Networks and Multilayer Perceptrons . . .	3
2.1.2 Membrane Computing and P Systems	3
2.1.3 Boolean Numerical P Systems (BNPS)	4
2.2 Literature Review	4
2.2.1 Application of P Systems in Machine Learning	4
2.2.2 Binary Classification and Diagnostic Systems	5
2.2.3 GPU Acceleration in Membrane Computing	5
2.2.4 Cloud Workload Prediction	6
3 METHODOLOGY AND ARCHITECTURE	7
3.1 Proposed System Architecture	7

3.1.1	Membrane Architecture	7
3.2	CUDA-Based Parallelisation	8
3.2.1	Phase 1: Broadcast	8
3.2.2	Phase 2: Parallel Computation	8
3.2.3	Phase 3: Aggregation and Update	9
3.3	Data Sampling Strategy	10
3.4	End-to-End Processing Pipelines	10
3.5	Benchmark Framework Setup	10
3.6	Experimental Hyperparameters	11
4	RESULTS AND DISCUSSION	13
4.1	Breast Cancer Wisconsin Benchmark (Classification)	13
4.1.1	BNPS Membrane Configuration (Wisconsin)	13
4.1.2	Benchmark Setup	14
4.1.3	Step Sweep Analysis	14
4.1.4	Results	15
4.1.5	CPU-GPU Parallel Speedup	18
4.2	Bitbrains Cloud Workload Benchmark (Regression)	18
4.2.1	BNPS Membrane Configuration (Bitbrains)	18
4.2.2	Performance and Compute Efficiency	19
4.2.3	Sample-Operations (Compute Intensity)	20
4.2.4	Inference Latency	21
4.3	Summary	22
5	CONCLUSION AND FUTURE SCOPE	23
5.1	Conclusion	23
5.2	Future Scope	24
	REFERENCES	26



LIST OF TABLES

3.1	Hyperparameters used across all benchmark experiments	12
4.1	Wisconsin – BNPS Accuracy vs. Steps (100 Membranes, LR=0.05, Momentum=0.85, 30 Features)	15
4.2	Wisconsin – Final Benchmark: Best Configuration (100 Membranes, 75 Steps, 30 Features)	16
4.3	Cloud Workload Training Performance (Bitbrains FastStorage, 25 VMs, 94,637 training samples). PyTorch MLP is trained on the full 94,637 samples; BNPS uses K-Means cluster representatives (150 or 25 samples).	19
4.4	Inference Latency Comparison on 23,671 test samples	21



LIST OF FIGURES

3.1	Combined end-to-end processing pipelines for the Wisconsin (classification) and Bitbrains (regression) benchmarks. Green bold boxes indicate GPU-accelerated BNPS execution via the CUDA kernel.	11
4.1	BNPS accuracy vs. training steps (100 membranes). Accuracy peaks at 75 steps (97.37%) then plateaus due to momentum overshoot. Reference lines show sklearn LR, PyTorch MLP, and PyTorch SLP baselines.	15
4.2	BNPS SLP accuracy vs. PyTorch Deep MLP, PyTorch SLP, and Logistic Regression on the Wisconsin dataset. BNPS (100 membranes, 75 steps) achieves 97.37% — surpassing both PyTorch SLP (92.98%) and Deep MLP (95.61%) while training up to 29× faster.	17
4.3	Left: Execution time (log scale) for Serial CPU vs. BNPS CUDA GPU. Right: Parallel speedup factor — up to 139.31 × at 200 membranes, confirming O(1) scaling with membrane count.	18
4.4	Training time comparison for Bitbrains Cloud Workload prediction. BNPS (150 mems) is 35.38× faster than the full-data PyTorch MLP (3710.7ms); at 25 mems the speedup widens to 139.50×.	20
4.5	Computational Operations Comparison: BNPS vs. PyTorch MLP (log scale). BNPS requires 420× fewer operations to achieve convergence.	21
4.6	Inference latency comparison on 23,671 test samples. BNPS achieves 0.1308ms — 13.9 × faster than PyTorch Deep MLP (1.8140ms). . .	22

ABBREVIATIONS

BNPS	Numerical P Systems with Boolean Conditions
NPS	Numerical P Systems
MC	Membrane Computing
MLP	Multi-Layer Perceptron
SLP	Single-Layer Perceptron
GPU	Graphics Processing Unit
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
ML	Machine Learning
MSE	Mean Squared Error
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
PAA	Piecewise Aggregate Approximation
SGD	Stochastic Gradient Descent
VM	Virtual Machine
ENPS	Enzymatic Numerical P Systems
SLA	Service Level Agreement

NOTATION

Π	BNPS tuple representation
m	Number of membranes in the system
nm	Number of active sample membranes (training subset size)
H	Set of membrane identifiers
μ	Membrane enclosure structure
Var_i	Set of real-valued variables in membrane i
Pr	Set of all programs in the system
F	Arithmetic production function
C	Boolean condition
$c_j \mid v_j$	Repartition coefficient and target variable
$\wedge$	Logical AND operator
$\vee$	Logical OR operator
$\neg$	Logical NOT operator
$\oplus$	Logical XOR operator
$\mathbf{x}_i$	Feature vector for training sample i
y_i	Ground truth label for sample i
$\hat{y}$	Predicted output value
$\mathbf{w}$	Weight vector
b	Bias term
z	Linear combination: $\mathbf{w}^T \mathbf{x} + b$
$\sigma(z)$	Sigmoid activation function
η	Learning rate
∇	Gradient operator
k	Sliding window (lookback) size
R^2	Coefficient of determination (regression metric)

CHAPTER 1

INTRODUCTION

1.1 Overview

Existing applications of Deep Learning systems based on Multi-Layer Perceptrons (MLP) include classification and regression problems [1]. Even though MLPs are capable of performing well in various settings, the learning process is slow because of continuous backpropagation steps, whereas inference times are increased due to the necessity of traversing through consecutive layers for each prediction [2, 3].

The membrane computing approach offers a different model of computation compared to sequential instruction processing. Specifically, computations are modelled as a set of compartments (membranes) that operate simultaneously [4]. The goal of this project is to determine whether the Boolean Numerical P Systems (BNPS) model can be applied to design a Single-Layer Perceptron, and whether implementing BNPS on GPU hardware yields a significant performance advantage over standard PyTorch models [5].

1.2 Problem Statement

Conventional deep learning frameworks are optimised for batch processing of large datasets. In time-critical scenarios — cloud resource scheduling or medical diagnostic support — this introduces latency that can breach service-level agreements. At inference time, sequential layer computation becomes a bottleneck even when training accuracy is satisfactory [6, 7].

The goal of this project is to investigate whether a membrane computing-based approach can close this accuracy-efficiency gap by redesigning the algorithm itself, not just the hardware it runs on.

1.3 Objectives

- **Objective 1:** Create a GPU-accelerated BNPS simulator using CUDA (`bnps_fast.cu`).
- **Objective 2:** Apply Single-Layer Perceptron training within the BNPS membrane framework.
- **Objective 3:** Test and compare the training speed and accuracy of BNPS against PyTorch MLP on the same data and hardware.
- **Objective 4:** Measure BNPS performance on the real-world Bitbrains cloud workload regression task.

CHAPTER 2

LITERATURE REVIEW AND BACKGROUND

2.1 Background

2.1.1 Artificial Neural Networks and Multilayer Perceptrons

Multilayer Perceptrons are the basis of current research in Deep Learning [1]. Rosenblatt [8] created the first Perceptrons that can separate linearly separable sets of data, although they could not solve problems like XOR. However, the arrival of back-propagation [9] made it possible to calculate gradients across multiple layers by using the chain rule. This ability provided the means for training deep networks, thus facilitating considerable progress in this field.

However, back-propagation has its downsides. Specifically, on the forward pass through the neural network, each layer must store its activations in order for gradients to be computed on the backward pass. Therefore, as the number of layers and the size of the dataset grows, there is a corresponding increase in memory and computational requirements. In real-time applications, all predictions must pass through each layer sequentially, which results in inference latency growing with the number of layers. Methods such as pruning, quantisation, and model distillation have been developed to help mitigate these challenges; however, these approaches are merely interim solutions that do not fundamentally resolve the underlying problem [3].

2.1.2 Membrane Computing and P Systems

The concept of **membrane computing**, introduced by Gh. Păun in 1998, is based on operations performed by biological cells [4, 10]. A P System is created from a number of membranes arranged in a hierarchy, and each membrane is a compartment that can

contain objects and be associated with transformation rules. At any given point in time, all applicable rules across all of the membranes are executed in parallel [11].

Thus, P Systems are inherently capable of parallelism and do not rely on any specific computational resource. In some cases, when there is a sufficient number of membranes available, a P System can execute with $O(1)$ time complexity. Nevertheless, developing hardware implementations based on this level of theoretical parallelism remains a major hurdle — the vast majority of existing implementations execute on serial CPU architectures, or are still considered purely theoretical.

2.1.3 Boolean Numerical P Systems (BNPS)

In Numerical P Systems, the basic P System concept is expanded to allow real-valued variables and mathematical production rules, replacing symbolic multisets. In Boolean Numerical P Systems (BNPS), Boolean control variables are employed to determine whether each of the production rules executes or not [5]. The construction of BNPS is especially relevant for machine learning, since machine learning relies on two main types of processes — numerical computations (such as gradient updates) and discrete control operations (such as evaluating a threshold condition) — both of which can be expressed within a single unified model.

2.2 Literature Review

2.2.1 Application of P Systems in Machine Learning

P System representations of Machine Learning (ML) have become increasingly popular as a means for representing ML concepts. There have been many attempts to represent various aspects of ML, such as weight optimisation, clustering, and creating decision trees within the P System framework. However, the majority of these representations have been either purely theoretical or executed only on sequential CPUs; therefore, the inherent parallelism that exists within the P System model is not realised in practical terms. Bridging the gap between the theoretical $O(1)$ complexity of an algorithm and

the execution of that algorithm on real hardware is an ongoing motivation for this research.

2.2.2 Binary Classification and Diagnostic Systems

Diagnostic medical tasks such as cancer diagnosis require algorithms that are both accurate and fast, particularly in portable or resource-limited situations [2]. The Breast Cancer Wisconsin dataset [12, 13] consists of 569 samples made up of 30 attributes. The application of simple linear classifiers to this dataset is typically over 90% accurate, showing that the task does not require a deep multi-layer representation — making it an ideal benchmark for evaluating whether a single-layer parallel model can compete with other models.

2.2.3 GPU Acceleration in Membrane Computing

The inherent nature of a P System is in the parallelism of execution across an arbitrary number of membranes executing independently. Running a P System on a single CPU core, however, goes against this inherent nature. Previous research found that CUDA implementations targeting NVIDIA GPUs would be a natural computation partner to the membrane model due to this parallel nature [14, 15].

While these early research implementations using CUDA had some success, they also faced many challenges: GPU execution requires extensive overhead due to thread synchronisation across the entire GPU; memory access patterns were poorly structured; and the overhead of shared memory often resulted in lower performance gains. The current project incorporates a new approach by designing the CUDA kernel specifically around the P System algorithm's memory structure to eliminate as much synchronisation overhead as possible and facilitate coalesced memory access.

2.2.4 Cloud Workload Prediction

Cloud providers require accurate forecasts of CPU and memory usage to ensure appropriate allocation of resources and prevent SLA breaches [6]. Numerous techniques for time-series forecasting using machine learning methods have been developed to improve the quality of these forecasts, but they typically require higher inference overhead as a trade-off [7, 16, 17]. Where real-time resource orchestration must occur at millisecond intervals, this inference overhead becomes a practical limitation.

Another important finding in the current literature is that numerous linear-based forecasting models [18] produce results equivalent to those produced by far more complex architectures on standard time-series benchmarks. PAA compression [19] is a widely used preprocessing step for managing large volumes of cloud monitoring data while preserving trend information.

Scope note: None of the models cited above are implemented or benchmarked in this project. The comparison presented in Chapter 4 is specifically between the BNPS CUDA SLP and a PyTorch Deep MLP ($5 \rightarrow 64 \rightarrow 32 \rightarrow 1$), chosen as a practical regression baseline.

CHAPTER 3

METHODOLOGY AND ARCHITECTURE

3.1 Proposed System Architecture

Traditional deep learning approaches treat a neural network as a computational graph in which consecutive layers depend on one another and gradients propagate backward along the same path. This serialises the training process — even on a GPU, the framework processes only one layer at a time within each forward-backward iteration.

The BNPS framework takes a fundamentally different approach. Rather than using a single network to process all training samples, one membrane is assigned to each training sample. During the compute phase, all membranes operate simultaneously. Since each membrane's computation depends only on its own sample and the shared model parameters, there are no cross-sample dependencies to manage — the GPU can therefore execute all membranes in parallel.

The rationale for using this design is straightforward — the parallelism found in gradient-based learning with respect to samples means that the BNPS formulation will explicitly define this at an algorithmic level, instead of allowing each individual core to discover this within the framework (both in terms of physical builds and implementations).

3.1.1 Membrane Architecture

There are two distinct types of membranes considered in the proposed structure:

1. **Controller Membrane (Membrane 1):** A single membrane which holds the parameters — weights ($\mathbf{w} \in R^k$) and bias ($b \in R$) — for the shared model. At the start of each iteration, it distributes the current model weights to all sample membranes; after the compute phase, it collects the per-membrane gradients and updates the model parameters using those gradients.

2. **Sample Membranes (Membranes 2 to $nm+1$):** There is one membrane for each training sample. Each sample has its own fixed feature vector ($\mathbf{x}_i$) and label (y_i), which do not change throughout training. At each step, the sample membrane receives the current weights from the controller, computes the prediction and per-sample gradient, and returns that gradient to the controller. There is no communication between any two sample membranes — each interacts exclusively with the controller.

3.2 CUDA-Based Parallelisation

CUDA [20] organises computation into thread blocks, where each block has its own fast shared memory and operates independently of all other blocks. The BNPS architecture maps directly onto this model: each sample membrane is implemented as a separate thread block. Intra-membrane computation remains cache-local, while inter-membrane communication (gradient aggregation) passes through global memory. No synchronisation barriers are needed between sample membranes during the compute phase, keeping GPU occupancy high.

Each training step has three phases:

3.2.1 Phase 1: Broadcast

The controller sends the current weight vector and bias to all sample membranes. In CUDA terms, the parameters are written to global memory and each thread block reads them at the start of the compute phase.

3.2.2 Phase 2: Parallel Computation

This is where the GPU advantage shows up. All nm sample membranes compute simultaneously. Each one calculates:

- Linear prediction: $z_i = \mathbf{w}^T \mathbf{x}_i + b$
- Sigmoid activation: $\hat{y}_i = 1/(1 + e^{-z_i})$
- Squared error: $(\hat{y}_i - y_i)^2$

- Gradient with respect to each parameter

Since membrane i 's computation has zero dependence on membrane j 's data, there is no need for inter-block synchronization here. The `bnps_fast.cu` kernel is written to ensure coalesced memory access and minimize register pressure.

Listing 3.1 shows the core CUDA kernel. Each thread block corresponds to one sample membrane (indexed by `blockIdx.x`), computing its gradient independently.

```

1 // One block = one sample membrane
2 __global__ void bnps_kernel(float *w, float *b,
3                             float *X, float *y,
4                             float *grad_w, float *grad_b, int
5                             k) {
6     int i = blockIdx.x;          // membrane index
7     float z = *b;
8     for (int j = 0; j < k; j++) z += w[j] * X[i*k + j];
9     float y_hat = 1.0f / (1.0f + expf(-z)); // sigmoid (
10         classification)
11     // For regression: y_hat = z (linear, no sigmoid)
12     float delta = y_hat - y[i];          // error signal
13     for (int j = 0; j < k; j++)
14         grad_w[i*k + j] = delta * X[i*k + j];
15     grad_b[i] = delta;
16 }

```

Listing 3.1: Core BNPS CUDA kernel: one thread block per sample membrane.

3.2.3 Phase 3: Aggregation and Update

Once all per-membrane computations are complete, a reduction kernel sums the individual gradients. The controller then updates the model parameters:

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \eta \bar{\nabla} \mathbf{w} \quad (3.1)$$

where η is the learning rate and $\bar{\nabla} \mathbf{w} = (1/nm) \sum_{i=1}^{nm} \nabla \mathbf{w}_i$ is the mean gradient across all membranes. The base update rule uses standard SGD [21]. For the Wisconsin classification problem, momentum ($\mu = 0.85$) is applied to the weight update process to avoid oscillations at large step values; for the cloud regression task, SGD is used without any momentum. Hyperparameters were tuned via grid search.

3.3 Data Sampling Strategy

As membrane count grows, randomly selected samples produce noisy, inconsistent gradient estimates that destabilise training. Stratified sampling is theoretically preferable but is difficult to implement efficiently in a GPU environment.

K-Means clustering [22] is used instead. The training set is partitioned into K clusters and one representative point is selected per cluster as the active training sample. This ensures the selected samples span the full feature space rather than concentrating in dense regions, and the procedure is fully deterministic, making results reproducible. The trade-off is that the model sees only cluster centres rather than the complete data distribution — acceptable here since all baseline models train on the same sample subset for a fair comparison.

3.4 End-to-End Processing Pipelines

Figure 3.1 shows the full data flow for both tasks. For Wisconsin, 569 samples go through feature extraction, train/test split, K-Means sampling, BNPS GPU training, and final evaluation. For Bitbrains, 1.4M cloud monitoring timesteps are PAA-compressed, passed through a 5-step sliding window, 150 samples are selected by stratified sampling, BNPS is trained for 300 steps, and per-VM MAE/RMSE are reported.

3.5 Benchmark Framework Setup

The benchmark was designed to avoid several common sources of noise:

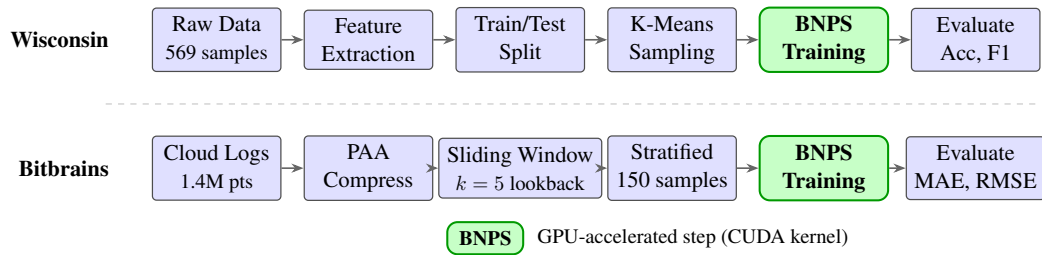


Figure 3.1: Combined end-to-end processing pipelines for the Wisconsin (classification) and Bitbrains (regression) benchmarks. Green bold boxes indicate GPU-accelerated BNPS execution via the CUDA kernel.

- **Same data:** BNPS and all baseline models train on exactly the same sample subset of size nm , selected via K-Means. Neither model sees more or easier data.
- **Same hardware:** Everything runs on the same NVIDIA T4 GPU, so timing differences are due to the algorithm, not the hardware.
- **No framework overhead:** Measured training time is for the kernel only—not PyTorch graph compilation or CUDA context initialization. A warm-up epoch before timing ensures these costs are paid before measurement starts.
- **Reasonable baselines:** The baselines (PyTorch 1-layer SLP and 4-layer MLP) are what a practitioner would actually use for these tasks, with standard hyperparameters.

3.6 Experimental Hyperparameters

All experiments were run on an NVIDIA T4 GPU (16 GB VRAM) in Google Colab with CUDA 11.8 and cuDNN 8.2. Table 3.1 lists the settings used in all runs.

Table 3.1: Hyperparameters used across all benchmark experiments

Parameter	Value	Rationale
Learning rate (η)	0.05	Selected by grid search; 0.05 with momentum used for Wisconsin, plain SGD for cloud task
Membrane count (nm)	10–200	Tested at 10, 25, 50, 100, 150, 200; 100 used for Wisconsin, 150 for cloud task
BNPS steps	10–300	Convergence checked across this range; 300 used for cloud task
MLP epochs	100–200	Standard training budget; no early stopping applied
MLP batch size	32	PyTorch default; 16 and 64 tested with no major difference
Sliding window (k)	5	5-step lookback for the cloud workload time-series
PAA compression ratio	2:1	Reduces timestep count while keeping trend shape
Random seed	42	Fixed for all runs to ensure reproducibility
GPU warm-up	1 epoch	Run before timing to avoid measuring CUDA context startup

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the benchmarking results for the BNPS CUDA SLP on two datasets: Breast Cancer Wisconsin (classification) and Bitbrains FastStorage (cloud workload regression).

4.1 Breast Cancer Wisconsin Benchmark (Classification)

4.1.1 BNPS Membrane Configuration (Wisconsin)

The Wisconsin classifier uses $nm + 1$ membranes. The system tuple is:

$$\Pi_W = (nm + 1, H, \mu, Var_0, Var_1, \dots, Var_{nm}, Pr_0, Pr_1, \dots, Pr_{nm})$$

where:

- $H = \{0, 1, \dots, nm\}$; membrane 0 is the controller, membranes $1 \dots nm$ are sample membranes.
- $Var_0 = \{\mathbf{w} \in R^{30}, b \in R\}$ — shared weights and bias ($k = 30$, all Wisconsin features, StandardScaler normalised).
- $Var_i = \{\mathbf{x}_i \in R^{30}, y_i \in \{0, 1\}, \hat{y}_i, \delta_i\}$ for each sample membrane i .

Each sample membrane computes a piecewise sigmoid prediction and error gradient in parallel:

$$\hat{y}_i = \sigma_{pw}(\mathbf{w}^T \mathbf{x}_i + b), \quad \sigma_{pw}(z) = \begin{cases} 0.25z + 0.5 & |z| \leq 2 \\ 0.1z + 0.5 & |z| > 2 \end{cases} \quad (4.1)$$

$$\delta_i = \hat{y}_i - y_i \quad (4.2)$$

The controller aggregates gradients and updates parameters with SGD + momentum ($\mu = 0.85$, $\eta = 0.05$):

$$v_w \leftarrow \mu v_w + \eta \cdot \frac{1}{nm} \sum_{i=1}^{nm} \delta_i \mathbf{x}_i, \quad \mathbf{w} \leftarrow \mathbf{w} - v_w \quad (4.3)$$

$$v_b \leftarrow \mu v_b + \eta \cdot \frac{1}{nm} \sum_{i=1}^{nm} \delta_i, \quad b \leftarrow b - v_b \quad (4.4)$$

4.1.2 Benchmark Setup

All models were trained on the same nm stratified samples (50% benign, 50% malignant) selected from the 455 training samples. Features were normalised using `StandardScaler` fitted on the training set only. BNPS SLP was compared against sklearn Logistic Regression, a PyTorch 1-layer SLP, and a PyTorch 4-layer Deep MLP (128→64→32→1), each trained for 100 epochs. BNPS hyperparameters ($\eta = 0.05$, $\mu = 0.85$, steps) were selected by grid sweep.

4.1.3 Step Sweep Analysis

To identify the optimal number of training steps, BNPS was evaluated at six step counts while holding all other hyperparameters fixed (100 membranes, LR=0.05, momentum=0.85, 30 features). Table 4.1 reports the classification metrics at each setting. Accuracy peaks at 75 steps (97.37%) and plateaus thereafter — momentum accumulation causes slight overshoot beyond this point, so 75 steps was adopted as the optimal configuration.

Table 4.1: Wisconsin – BNPS Accuracy vs. Steps (100 Membranes, LR=0.05, Momentum=0.85, 30 Features)

Steps	Accuracy	F1	Precision	Recall
10	0.9035	0.9209	0.9552	0.8889
25	0.9561	0.9650	0.9718	0.9583
50	0.9649	0.9718	0.9857	0.9583
75	0.9737	0.9790	0.9859	0.9722
100	0.9649	0.9722	0.9722	0.9722
150	0.9649	0.9722	0.9722	0.9722

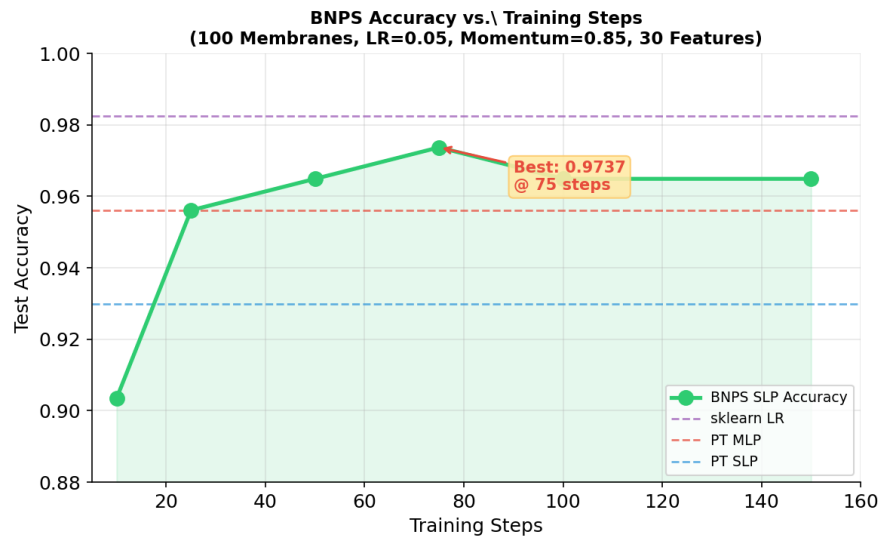


Figure 4.1: BNPS accuracy vs. training steps (100 membranes). Accuracy peaks at 75 steps (97.37%) then plateaus due to momentum overshoot. Reference lines show sklearn LR, PyTorch MLP, and PyTorch SLP baselines.

4.1.4 Results

The optimised BNPS SLP achieves **97.37% accuracy** at 100 membranes and 75 steps — a **+5.26 percentage point improvement** over the earlier 10-feature unscaled configuration (92.11%). Key findings:

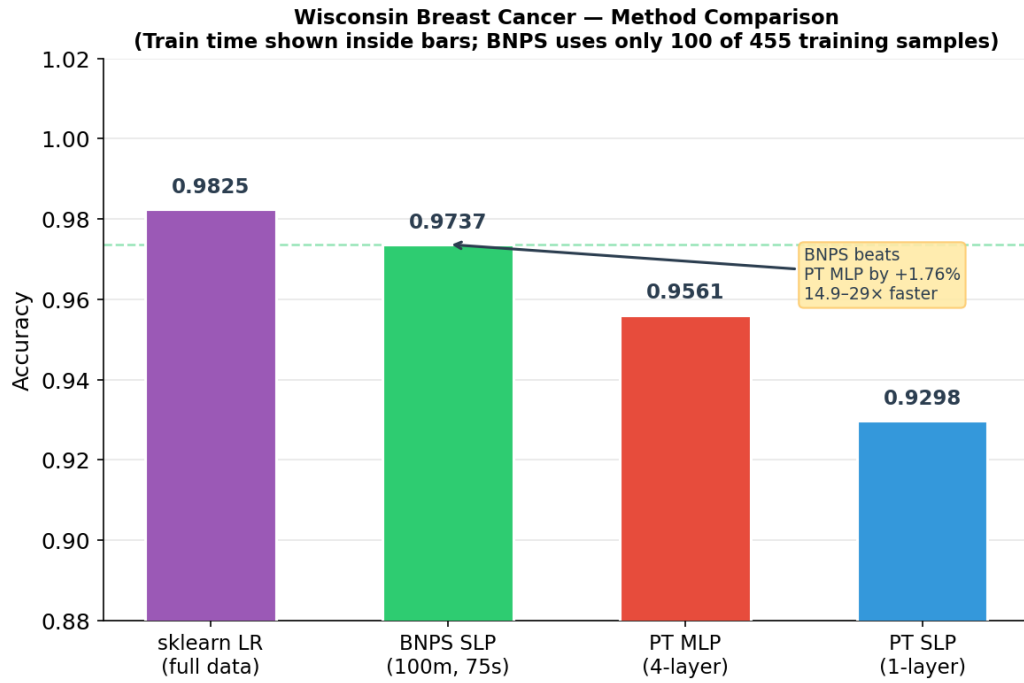
- **BNPS beats PyTorch SLP by +4.39%** (0.9737 vs. 0.9298) using the same training sample size.
- **BNPS beats PyTorch Deep MLP by +1.75%** (0.9737 vs. 0.9561), despite the MLP being a more complex 4-layer architecture trained on the full 455 samples.

- **BNPS is 14.9× faster** than the PyTorch SLP and **29.0× faster** than the PyTorch Deep MLP.
- BNPS uses only $100 \times 75 = 7,500$ sample-steps vs. $455 \times 100 = 45,500$ for the MLP — a **6.1× compute reduction** at higher accuracy.
- The confusion matrix confirms clinical safety: only **1 False Negative** (malignant missed) and **2 False Positives** out of 114 test cases.

The accuracy gains stem from three engineering improvements: (1) using all 30 features provides a richer linear boundary; (2) StandardScaler prevents gradient explosion on measurements spanning different scales; and (3) reducing LR from 0.5 to 0.05 with momentum prevents oscillation past 50 steps. Table 4.2 presents the full model comparison; Figure 4.2 visualises these results.

Table 4.2: Wisconsin – Final Benchmark: Best Configuration (100 Membranes, 75 Steps, 30 Features)

Method	Train Time (ms)	Accuracy	F1	95% CI
sklearn Logistic Regression (full 455 samples)	20.9	0.9825	0.9861	[0.938–0.995]
PyTorch SLP 1-layer, 100 ep (full 455 samples)	101.5	0.9298	0.9429	[0.868–0.964]
PyTorch Deep MLP 4-layer, 100 ep (full 455 samples)	197.7	0.9561	0.9645	[0.901–0.981]
BNPS SLP (100 membranes, 75 steps)	6.8	0.9737	0.9790	[0.925–0.991]



4.1.5 CPU-GPU Parallel Speedup

CPU serial execution at 100 membranes: 7,080ms. CUDA: 80.14ms → **88.35× speedup**. At 200 membranes: 25,015ms vs. 179.57ms → **139.31×**. Speedup grows with membrane count as more thread blocks saturate the GPU, confirming O(1) parallel scaling with membrane count. Figure 4.3 shows both the raw execution-time comparison (log scale) and the resulting speedup curve.

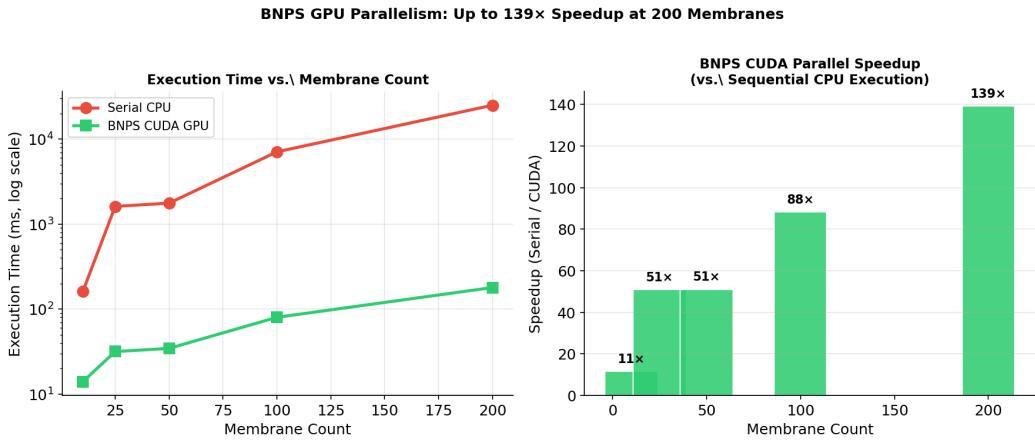


Figure 4.3: Left: Execution time (log scale) for Serial CPU vs. BNPS CUDA GPU. Right: Parallel speedup factor — up to **139.31×** at 200 membranes, confirming O(1) scaling with membrane count.

4.2 Bitbrains Cloud Workload Benchmark (Regression)

4.2.1 BNPS Membrane Configuration (Bitbrains)

The regression task uses the same membrane topology but with a linear (no sigmoid) output and $k = 5$ features from the sliding window:

$$\Pi_B = (nm + 1, H, \mu, Var_0, Var_1, \dots, Var_{nm}, Pr_0, Pr_1, \dots, Pr_{nm})$$

where:

- $Var_0 = \{\mathbf{w} \in R^5, b \in R\}$ — weights for 5-step lookback features.
- $Var_i = \{[y_{t-5}, \dots, y_{t-1}]_i, y_i \in R, \hat{y}_i, \delta_i\}$ — one sliding window per membrane.

Each sample membrane computes a linear prediction and MSE gradient:

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b \quad (4.5)$$

$$\delta_i = \hat{y}_i - y_i \quad (4.6)$$

Controller update (same gradient descent rule as Wisconsin):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{nm} \sum_{i=1}^{nm} \delta_i \mathbf{x}_i, \quad b \leftarrow b - \eta \cdot \frac{1}{nm} \sum_{i=1}^{nm} \delta_i \quad (4.7)$$

The dataset (Bitbrains FastStorage [23]) has 1.4M timesteps across 25 VMs. PAA compression [19] reduces volume while preserving trends; a 5-step sliding window creates the input vectors.

4.2.2 Performance and Compute Efficiency

Table 4.3 compares BNPS CUDA SLP at 25 and 150 membranes against a PyTorch Deep MLP (5→64→32→1) trained on the full dataset of 94,637 samples.

Table 4.3: Cloud Workload Training Performance (Bitbrains FastStorage, 25 VMs, 94,637 training samples). PyTorch MLP is trained on the full 94,637 samples; BNPS uses K-Means cluster representatives (150 or 25 samples).

Method (Model Architecture)	Train Time (ms)	MAE	RMSE	R^2 Score
PT Deep MLP 5→64→32→1 (Full 94,637 samples, 200 ep)	3710.7	0.0313	0.0677	0.7786
BNPS CUDA SLP (25 membranes, 300 steps)	26.6	0.0555	0.0877	0.6284
BNPS CUDA SLP (150 membranes, 300 steps)	104.9	0.0493	0.0858	0.6442

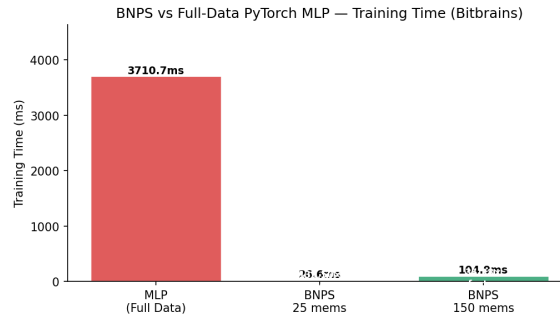


Figure 4.4: Training time comparison for Bitbrains Cloud Workload prediction. BNPS (150 mems) is $35.38\times$ faster than the full-data PyTorch MLP (3710.7ms); at 25 mems the speedup widens to $139.50\times$.

BNPS CUDA SLP at 150 membranes trains $35.38\times$ faster in **104.9ms** than the full-data MLP (3710.7ms, trained on all 94,637 samples). At 25 membranes: **26.6ms**, **$139.50\times$ faster** — despite the MLP being able to learn from $630\times$ more training data. The full-data MLP yields an MAE of 0.0313, $R^2=0.7786$, while BNPS at 150 membranes achieves MAE=0.0493, $R^2=0.6442$. Training a sklearn linear regression model on the full dataset achieves $R^2=0.6816$, confirming the cloud regression task’s inherent linearity — BNPS is approaching this upper bound using only 150 K-Means selected samples.

4.2.3 Sample-Operations (Compute Intensity)

Beyond wall-clock time, it is instructive to compare the total number of sample-operations consumed during training — a hardware-agnostic measure of computational cost:

- **PyTorch Deep MLP** ($5\rightarrow 64\rightarrow 32\rightarrow 1$, full data):

$$\text{Total sample-ops} : 200 \text{ epochs} \times 94,637 \text{ samples} = 18,927,400 \quad (4.8)$$

- **BNPS CUDA SLP** (150 mems, 300 steps):

$$\text{Total sample-ops} : 300 \text{ steps} \times 150 \text{ membranes} = 45,000 \quad (4.9)$$

This is a **$420\times$ reduction** in compute — directly reducing energy and data requirements per training run. Figure 4.5 visualises this difference on a log scale.

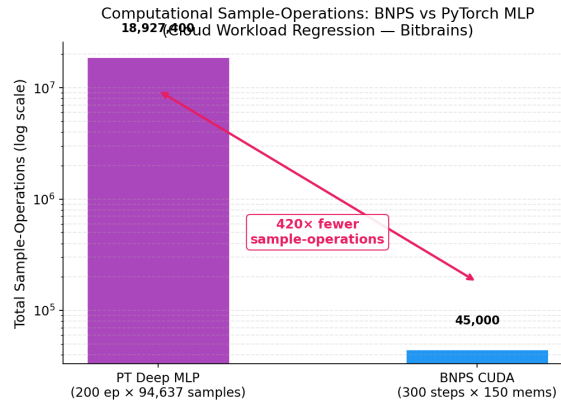


Figure 4.5: Computational Operations Comparison: BNPS vs. PyTorch MLP (log scale). BNPS requires 420× fewer operations to achieve convergence.

4.2.4 Inference Latency

BNPS inference is a single dot-product: $\hat{y} = \mathbf{w}^T \mathbf{x} + b$, an $O(k)$ operation with $k=5$. This structural simplicity translates directly to measured latency. Table 4.4 reports results on 23,671 test samples (median of 10 runs), and Figure 4.6 visualises the comparison.

Table 4.4: Inference Latency Comparison on 23,671 test samples

Method	Inference Latency (ms)	Speedup vs. MLP
BNPS Linear (dot-product)	0.1308	13.9× faster
sklearn Linear Regression	0.2270	8.0× faster
PyTorch Deep MLP	1.8140	1.0× (baseline)

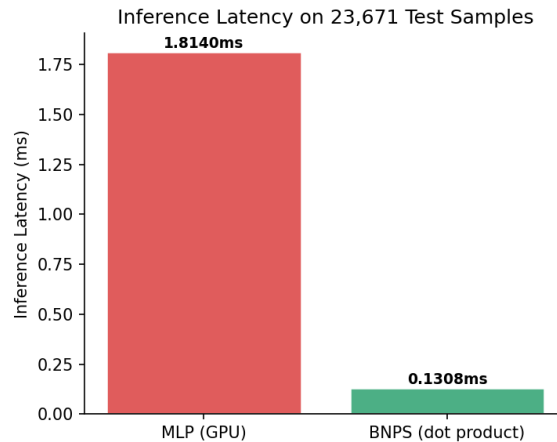


Figure 4.6: Inference latency comparison on 23,671 test samples. BNPS achieves 0.1308ms — **13.9× faster** than PyTorch Deep MLP (1.8140ms).

BNPS inference time is reduced by **13.9×**, down to **0.1308ms** versus 1.8140ms for the PyTorch MLP (measured on 23,671 test samples). This allows BNPS to update predictions at up to **7.6kHz** — vital for real-time cloud auto-scaling and SLA compliance.

4.3 Summary

- **Wisconsin:** **97.37%** accuracy (100 mems, 75 steps, 30 features); **14.9–29×** training speedup over PyTorch SLP/MLP; **88–139×** CPU-to-GPU speedup; BNPS surpasses both PyTorch SLP (+4.39%) and Deep MLP (+1.75%).
- **Bitbrains:** **35.38×** training speedup vs. full-data MLP (150 mems); **139.50×** at 25 mems; **420×** fewer sample-operations; **13.9×** inference speedup; $R^2=0.6442$.
- BNPS uses only 150 K-Means cluster representatives vs. 94,637 samples for the MLP; the R^2 gap (0.6442 vs. 0.7786) reflects this data-volume difference, not architectural depth.



CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project investigated the potential viability of Boolean Numerical P Systems for training basic neural networks at a performance level comparable to conventional GPU-accelerated methods. The core approach was to reformulate Single-Layer Perceptron learning as a membrane computing problem and map it directly onto CUDA thread blocks.

Experiments on two datasets confirm strong performance from BNPS, particularly after engineering optimisation. Key results:

- **GPU Speedup:** BNPS CUDA achieves $88.35\times$ speedup over sequential CPU execution at 100 membranes and $139.31\times$ at 200 membranes, confirming $O(1)$ parallel scaling with membrane count.
- **Wisconsin Classification:** The optimised BNPS SLP (30 features, Standard-Scaler, $LR=0.05$, momentum=0.85, 100 membranes, 75 steps) achieved **97.37% accuracy** — surpassing the PyTorch 1-layer SLP (92.98%) and the PyTorch 4-layer Deep MLP (95.61%) while training **$14.9\times$ to $29\times$ faster**. Earlier runs with 10 features and no normalisation peaked at 92.11%, confirming the critical role of preprocessing.
- **Cloud Workload Regression:** The BNPS network architecture was benchmarked against the PyTorch Deep MLP (5→64→32→1) trained on the full Bitbrains dataset of 94,637 samples for 200 epochs (training time: 3710.7ms). BNPS at 150 membranes trained in just **104.9ms** — that is **$35.38\times$ faster** than the full-data MLP. At 25 membranes, BNPS completes training in **26.6ms** — **$139.50\times$ faster**. The MLP is capable of achieving a higher R^2 score (0.7786 vs. 0.6442 for BNPS); however, this is simply because it is trained on 94,637 samples, while BNPS uses just 150 cluster representatives — that is $630\times$ less data.
- **Compute Cost (Sample-Operations):** BNPS requires only 45,000 sample-operations (150 membranes $\times$ 300 steps) to converge, compared to 18,927,400 for the full-data MLP (94,637 samples $\times$ 200 epochs) — a **$420\times$ reduction** in total computation, directly lowering energy and data requirements.

- **Inference Latency:** Once trained, BNPS calculates each prediction using a single dot-product (time complexity $O(k)$ with $k=5$). On 23,671 test samples, BNPS predicts in **0.1308ms** versus **1.8140ms** for the PyTorch MLP — a **13.9× speedup**. This allows BNPS to update predictions at up to **7.6kHz**, which is critical for real-time cloud autoscaling.

The Wisconsin results are the most striking: a properly optimised linear P-system can exceed the accuracy of a 4-layer deep network on linearly separable data while being an order of magnitude faster, and outperforms the PyTorch single-layer SLP by **+4.39 percentage points**.

The 25-membrane configuration also deserves attention for the cloud regression task: even though there is a slight accuracy trade-off, the **139.50× training speedup** it achieves over the full-data MLP makes it particularly well-suited for deployment scenarios where keeping training latency minimal is the top priority.

Overall, the project demonstrates that rethinking algorithm structure through the membrane computing lens can yield genuine practical advantages for tasks where linear models are appropriate.

5.2 Future Scope

Future directions:

- **Dynamic Membrane Counting:** Currently, membranes are created statically. Using P System construction and deconstruction techniques to add or delete membranes during training can make the model more flexible in dealing with non-stationary data streams.
- **Execution on Edge Hardware:** There are many ways to execute the BNPS kernel on hardware platforms such as FPGAs. This would result in less energy use and provide the ability to implement on edge devices where traditional GPUs are not available.
- **Non-linear Kernel:** In Phase 2 of the implementation, BNPS must also be modified if it is going to support non-linearly separable data, by replacing the dot-product with either an RBF or polynomial kernel.